PSG COLLEGE OF TECHNOLOGY
DEPARTMENT OF COMPUTER APPLICATIONS
23MX26 - Java Programming Laboratory
Practice Hands-on Worksheet 11 Exception Handling

Exception handling is the process of responding to the occurrence, during computation, of exceptions – anomalous or exceptional conditions requiring special processing – often changing the normal flow of program execution. (Wikipedia)

Java has built-in mechanism to handle exceptions. Using the *try* statement we can test a block of code for errors. The *catch* block contains the code that says what to do if exception occurs.

This problem will test your knowledge on try-catch block.

You will be given two integers and as input, you have to compute . If and are not bit signed integers or if is zero, exception will occur and you have to report it. Read sample Input/Output to know what to report in case of exceptions.

Sample Input 0:

10
3

Sample Output 0:

3

Sample Input 1:

10
Hello

Sample Output 1:

java.util.InputMismatchException

Sample Input 2:

10
0

Sample Output 2:

java.lang.ArithmeticException: / by zero

Sample Input 3:

23.323
0

Sample Output 3:

java.util.InputMismatchException

2. You are required to compute the power of a number by implementing a calculator. Create a class *MyCalculator* which consists of a single method long power(int, int). This method takes two integers, and , as parameters and finds . If either or is negative, then the method must throw an exception which says "". Also, if both and are zero, then the method must throw an exception which says ""

For example, *-4* and *-5* would result in .

Complete the function power in class *MyCalculator* and return the appropriate result after the power operation or an appropriate exception as detailed above.

Input Format

Each line of the input contains two integers, and . The locked stub code in the editor reads the input and sends the values to the method as parameters.

Constraints

-
-

Output Format

Each line of the output contains the result , if both and are positive. If either or is negative, the output contains "n and p should be non-negative". If both and are zero, the output contains "n and p should not be zero.". This is printed by the locked stub code in the editor.

Sample Input 0

3 5
2 4
0 0
-1 -2
-1 3

Sample Output 0

```
243
16
java.lang.Exception: n and p should not be zero.
java.lang.Exception: n or p should not be negative.
java.lang.Exception: n or p should not be negative.
```

Explanation 0

- In the first two cases, both and are postive. So, the power function returns the answer correctly.
- In the third case, both and are zero. So, the exception, "n and p should not be zero.", is printed.
- In the last two cases, at least one out of and is negative. So, the exception, "n or p should not be negative.", is printed for these two cases.